

Échantillonnage et estimation

Région Nouvelle-Aquitaine

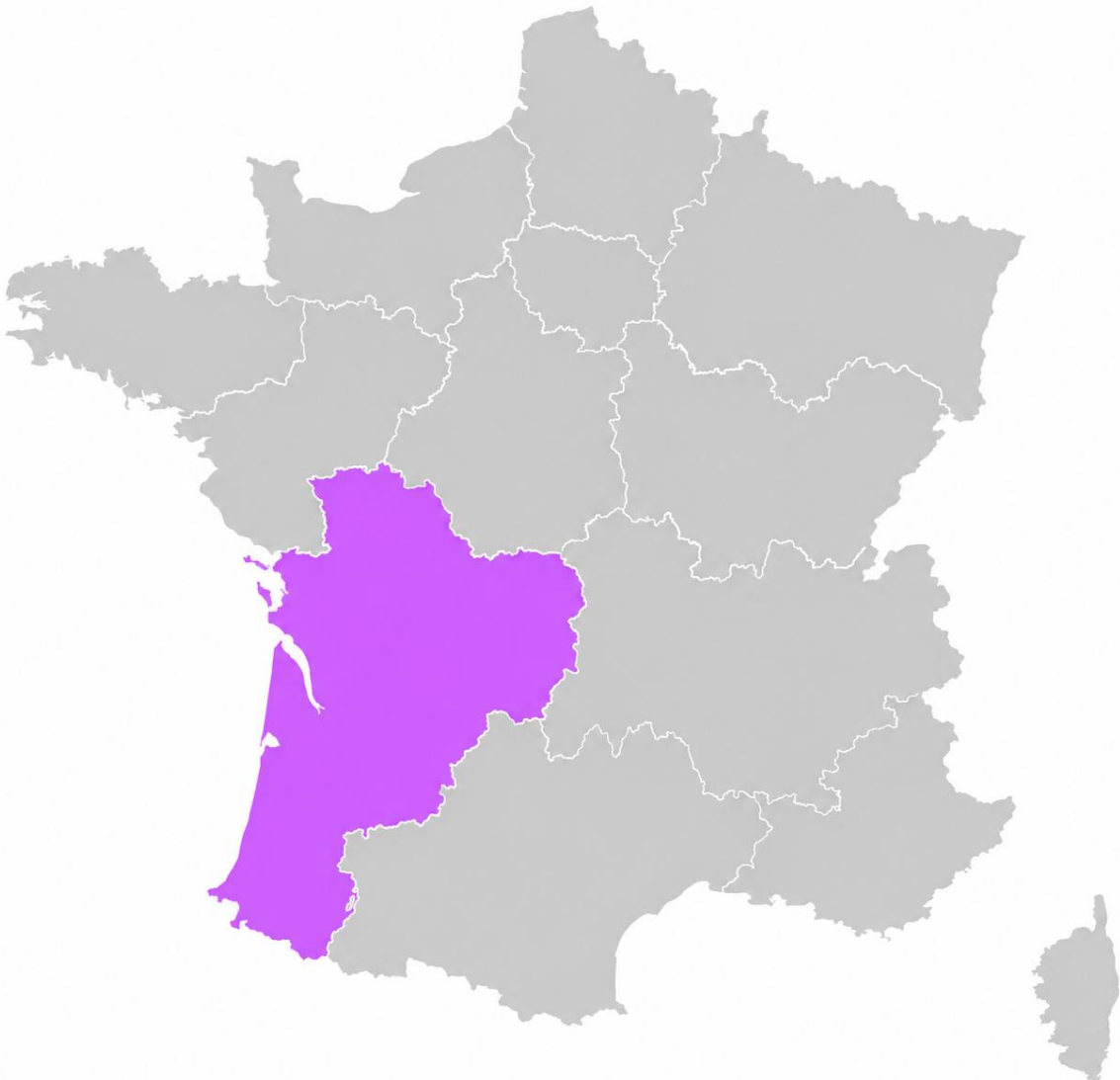


Table des matières :

Table des matières :.....	2
INTRODUCTION :	3
ESTIMATION DU NOMBRE D'HABITANTS EN NOUVELLE-AQUITAINE	3
Échantillonnage aléatoire simple	3
Échantillonnage aléatoire stratifié	7
Conclusion.....	9
Traitement de données d'enquête.....	10
Test d'indépendance du khi-deux.....	10
Conclusion.....	11
Code R	12
Échantillonnage aléatoire simple	12
Échantillonnage aléatoire stratifié	13
Test d'indépendant du khi-deux	15

INTRODUCTION :

Cette SAÉ a pour objectif de comprendre comment estimer une valeur à partir d'un échantillon, ainsi que de mesurer la précision de cette estimation grâce à un intervalle de confiance, et enfin d'analyser les relations possibles entre différentes variables statistiques à l'aide du test du khi-deux.

Dans une première partie, nous nous sommes intéressés à la population de la région Nouvelle-Aquitaine. Nous avons utilisé le logiciel R afin de réaliser un sondage aléatoire simple. Cette méthode consiste à sélectionner des individus au hasard, ici des communes, chacune d'entre elles ayant la même probabilité d'être choisie. À partir des données obtenues, nous avons calculé des estimations de la population et étudié leur précision.

Ensuite, nous avons appliqué une méthode de sondage stratifié. Cette technique consiste à diviser la population en plusieurs groupes appelés strates, puis à effectuer un tirage dans chacun de ces groupes. Nous avons ensuite comparé les résultats obtenus aux valeurs réelles afin d'évaluer la qualité et la précision des différentes méthodes de sondage.

Dans une seconde partie, nous allons utiliser le test du khi-deux afin d'étudier l'existence d'un lien entre la pratique du sport et différentes variables associées. Cette analyse nous permettra de vérifier si certaines variables sont dépendantes ou indépendantes les unes des autres.

ESTIMATION DU NOMBRE D'HABITANTS EN NOUVELLE-AQUITAINE

Échantillonnage aléatoire simple

Dans cette partie, nous allons estimer le nombre d'habitants de notre région à partir d'un échantillon de communes contenues dans un fichier issu d'un fichier de l'INSEE. Pour cela, nous utiliserons un échantillonnage aléatoire simple, où chaque commune possède la même probabilité d'être sélectionnée. Nous calculerons ensuite une estimation de la population totale ainsi qu'un intervalle de confiance afin d'évaluer la précision des résultats obtenus.

Tout d'abord nous nous situons dans le répertoire de travail, nous installons et chargeons la librairie sampling qui va permettre d'effectuer les différents tirages d'échantillons, puis nous importons notre csv contenant les données.

```
# Définition du répertoire
setwd("E:\\SAE Echantillonnage et estimation2")

# Chargement de la bibliothèque sampling
library(sampling)

# Lecture du fichier CSV
tableau <- read.csv2("population_francaise_communes.csv")
```

A présent nous devons trier nos données, nous effectuons donc un filtre sur la variable « Nom.de.la.région » afin de garder seulement celle de la Nouvelle-Aquitaine. Nous gardons uniquement les colonnes comprenant le code du département, le nom de la commune et sa population totale que nous stockons dans le data frame « donnees »

```
# Lecture du fichier CSV
tableau <- read.csv2("population_francaise_communes.csv")

# Extraction des communes de Nouvelle-Aquitaine avec les colonnes utiles
donnees <- tableau[tableau$Nom.de.la.région == "Nouvelle-Aquitaine",
  c("Code.département", "Commune", "Population.totale")]
```

Voici les 6 premières communes du data frame « donnee » :

```
#Affiche les 6 premières communes
head(donnees)
```

Code.département	Commune	Population.totale
16	Abzac	821
16	Les Adjots	542
16	Agris	869
16	Aigre	1615
16	Alloue	480
16	Ambérac	308

Ensuite, nous créons un vecteur U qui répertorie l'ensemble des noms des communes de la Nouvelle-Aquitaine se trouvant dans la colonne « Commune » du data frame. Nous calculons ainsi la taille de la population, représenté par le nombre de communes dans U, que nous stockons dans la variable N. Dans la variable T nous calculons et stockons le nombre total d'habitants de la région en faisant la somme de toute la colonne « Population.totale ».

```
# U : vecteur des noms de communes
U <- donnees$Commune

# N : taille de la population (nombre total de communes)
N <- length(U)

# T : total réel de la population
T <- sum(donnees$Population.totale)
```

Nous savons à présent qu'il y a 4309 communes en Nouvelle-Aquitaine et que la région compte 6 575 230 habitants.

Par la suite, on réalise un tirage aléatoire de 100 communes. La méthode est le tirage aléatoire simple sans remise avec la fonction « sample » appliquée au vecteur U avec une taille de 100. Ainsi, chaque commune possède une probabilité égale d'être sélectionnée. Puis on crée un data frame avec les données des communes de E via un filtre.

```
#Tirage d'un échantillon aléatoire simple sans remise
```

```
# Sélection aléatoire de n = 100 communes parmi U
```

```
n <- 100
```

```
E <- sample(U, n)
```

```
#Récupération des lignes correspondant aux communes de l'échantillon E
```

```
donnees1 <- donnees[donnees$Commune %in% E, ]
```

Sur ce tirage, on calcule la moyenne de la population des communes de l'échantillon ($\bar{x}$). Cette moyenne représente une estimation de la population moyenne d'une commune de la région Nouvelle-Aquitaine. On calcule ensuite un intervalle de confiance à 95 % de cette moyenne (idcmoy) grâce au test de Student ($t.\text{test}$). Cet intervalle donne une zone dans laquelle la vraie moyenne de la population des communes a 95 % de chances de se situer. À partir de cette moyenne, on estime ensuite le nombre total d'habitants de la région en la multipliant par le nombre de communes de la région (N). Cette valeur (T_{est}) constitue donc une estimation du total réel de population. L'intervalle de confiance du total (idcT) est obtenu en multipliant les bornes de l'intervalle de confiance de la moyenne par N . Il fournit un encadrement plausible du nombre total réel d'habitants de la région. Enfin, on calcule la marge d'erreur qui mesure la précision de l'estimation obtenue. Plus elle est faible, plus l'estimation du total de population est précise et proche de la réalité. À l'inverse, une marge d'erreur élevée indique une plus grande incertitude liée au tirage de l'échantillon.

```
#Estimation de la moyenne et intervalle de confiance
```

```
#Moyenne de la population totale sur l'échantillon (valeurs manquantes ignorées)
```

```
xbar <- mean(donnees1$Population.totale, na.rm = TRUE)
```

```
#Intervalle de confiance à 95 % de la moyenne via le test t
```

```
idcmoy <- t.test(donnees1$Population.totale)$conf.int
```

```
#Estimateur du total : moyenne de l'échantillon x taille de la population
```

```
T_est <- N * xbar
```

```
#Transposition de l'IDC de la moyenne en IDC du total (multiplication par N)
```

```
idcT <- idcmoy * N
```

```
#Marge d'erreur
```

```
marge <- (idcT[2] - idcT[1]) / 2
```

On souhaite refaire cette expérience 10 fois et de stocker les résultats obtenus dans un CSV, pour cela nous allons créer une boucle et refaire les mêmes opérations que précédemment

```
#Initialisation du tableau de résultats
```

```
resultats <- data.frame(  
  Pop_exacte = numeric(),  
  Pop_est    = numeric(),  
  IDC_inf    = numeric(),  
  IDC_sup    = numeric(),  
  Marge      = numeric()  
)
```

```
#Tirage des 10 échantillons
```

```
for (i in 1:10) {
```

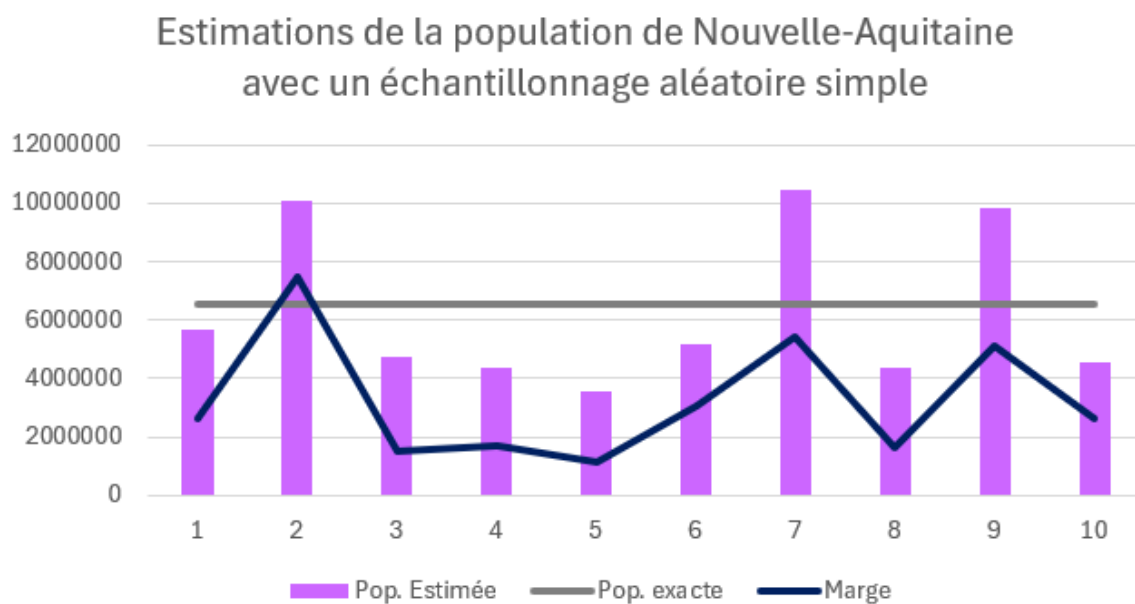
```

#Ajout d'une ligne au tableau : total réel, total estimé, marge d'erreur
new_row <- data.frame(Pop_exacte = T,
                      Pop_est = T_est,
                      Marge = marge,
                      IDC_inf = idcT[1],
                      IDC_sup = idcT[2])
resultats <- rbind(resultats, new_row)
}

#Export des résultats dans un fichier CSV (séparateur point-virgule)
write.csv2(resultats, "resultats1.csv", row.names = FALSE)

```

Grace a ce fichier, nous avons réalisé un graphique illustrant nos résultats :



Ce graphique représente un principe clé de l'échantillonnage aléatoire simple : tirer seulement 10 individus au hasard dans une population ne suffit pas à produire des estimations stables. Chaque tirage étant indépendant et sans structure, les résultats peuvent s'écarter très largement de la réalité, 3 des 10 échantillons gonflent la population à près environ 10 millions (10M) atteignant les 10.5M d'habitants alors que la vraie valeur est de 6,5M, tandis qu'à l'opposé, certains échantillons la sous-estiment à environ 4M, descendant jusqu'à 3.7M pour l'échantillon 5.

Ainsi, le sondage aléatoire simple montre certaines limites dans ce contexte. En effet, la région Nouvelle-Aquitaine est composée d'une grande majorité de petits villages et communes rurales. Ainsi, si une grande ville comme Bordeaux venait à être tirée, celle-ci va surreprésentée l'échantillon, ses valeurs plus élevées tireraient la moyenne vers le haut, faussant ainsi l'estimation.

Échantillonnage aléatoire stratifié

On effectue dans un premier temps le même tri que précédemment (on garde que les communes de la région) et on garde les mêmes colonnes. Afin de réaliser les meilleures strates possibles on utilise la fonction « summary » qui va nous indiquer les extrémités, les quartiles, la moyenne et la médiane de l'ensemble des populations de chaque communes.

```
#Résumé statistique de la variable Population.totale (min, max, médiane, moyenne, quartiles)  
summary(donnees$Population.totale)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
32	239	507	1526	1112	263247

Pour avoir des estimations les plus proche de la réalité, nous avons choisi de réaliser 9 strates avec : 12.5% des communes dans les 7 premières strates puis 11.5% dans la 8^{ème} et 1% dans la dernière strate. Cette dernière strate regroupe les communes avec une forte population (supérieure à 20 000). On récupère le tout dans le data frame « donneesstrat »

```
#Création de la variable de stratification par découpage en 9 strates
```

```
#Selon des seuils de population définis manuellement (découpage en quantiles ou expert)  
donnees$strate <- cut(donnees$Population.totale,  
                      breaks = c(0, 163, 239, 349, 507, 720, 1112, 2185, 20000, Inf),  
                      labels = c(1,2,3,4,5,6,7,8,9))
```

```
#Réduction du tableau aux colonnes utiles : commune, population et strate  
donneesstrat <- donnees[, c("Commune", "Population.totale", "strate")]
```

Puis on calcule le nombre de commune par strate et le poids de la strate afin de savoir combien de commune doit on tirer dans les strates.

```
#Paramètres de l'échantillonnage
```

```
#Trie par strate (obligatoire pour strata())  
data <- donneesstrat[order(donneesstrat$strate),]
```

```
#Nombre de communes par strate  
Nh <- table(data$strate)
```

```
#Nombre total de communes  
N <- sum(Nh)
```

```
#Poids de chaque strate  
gh <- Nh/N
```

Une fois que nous avons défini nos strates, nous procédons aux 10 répétitions de ses estimations à travers 10 échantillons distincts. Par la suite on les mettra dans un fichier Excel pour pouvoir réaliser un graphique à partir de la population exacte, de la population estimée et la marge.

Nous commençons donc en calculant le nombre de communes que doit contenir chaque strate avec au moins une commune par strate. Puis nous calculons les moyennes et variances par strate. Pour ne pas biaiser nos estimations, on réduit la variance à 0 pour la 9^e strate. Ensuite, nous calculons la moyenne et la variance de l'échantillon, toutes deux pondérées. Et enfin nous déterminons la population estimée de la Nouvelle-Aquitaine ainsi qu'un intervalle de confiance et la marge d'erreur de chacune des 10 estimations.

#Boucle : 10 tirages indépendants

```
for (i in 1:10) {
```

```
  #Taille souhaitée de l'échantillon  
  n <- 100
```

```
  #Calcul du nombre de communes par strate (pmax(..., 1) garantit au moins 1 unité tirée par strate)  
  nh <- pmax(round(c(n*Nh[1]/N, n*Nh[2]/N, n*Nh[3]/N, n*Nh[4]/N,  
                    n*Nh[5]/N, n*Nh[6]/N, n*Nh[7]/N, n*Nh[8]/N, n*Nh[9]/N)), 1)
```

```
  #Strate 9 exhaustive : toutes les grandes villes sont incluses (sert à ne pas avoir une variance biaisée)  
  nh[9] <- Nh[9]
```

```
  #Taux de sondage par strate = nh / Nh  
  fh <- nh/Nh
```

```
  #Tirage stratifié avec remise dans chaque strate  
  st <- strata(data, stratanames = c("strate"), size = nh, method = "srswr")
```

```
  #Extrait les données des communes tirées  
  data1 <- getdata(data, st)
```

```
  #Extraction des sous-échantillons par strate  
  ech1 <- data1[data1$strate==1, ]  
  ...
```

```
  #Moyennes de population par strate  
  m1 <- mean(ech1$Population.totale)  
  ...
```

```
  #Variances de population par strate  
  var1 <- var(ech1$Population.totale)  
  ...
```

```
  #Variance = 0 si 1 seul élément  
  var9 <- if (length(ech9$Population.totale) < 2) 0 else var(ech9$Population.totale)
```

```
  #Moyenne stratifiée = somme pondérée des moyennes par strate  
  Xbarst <- (Nh[1]*m1 + Nh[2]*m2 + Nh[3]*m3 + Nh[4]*m4 + Nh[5]*m5 +  
            Nh[6]*m6 + Nh[7]*m7 + Nh[8]*m8 + Nh[9]*m9) / N
```

```
  #Variance de l'estimateur : somme des contributions de chaque strate  
  varXbarst <- (gh[1]^2)*(1-fh[1])*var1/nh[1] + (gh[2]^2)*(1-fh[2])*var2/nh[2] +  
              (gh[3]^2)*(1-fh[3])*var3/nh[3] + (gh[4]^2)*(1-fh[4])*var4/nh[4] +  
              (gh[5]^2)*(1-fh[5])*var5/nh[5] + (gh[6]^2)*(1-fh[6])*var6/nh[6] +  
              (gh[7]^2)*(1-fh[7])*var7/nh[7] + (gh[8]^2)*(1-fh[8])*var8/nh[8] +  
              (gh[9]^2)*(1-fh[9])*var9/nh[9]
```

```
  #Total estimé = N * moyenne stratifiée  
  Tstr <- N * Xbarst  
  Tstr
```

#Intervalle de confiance à 95% pour le total T

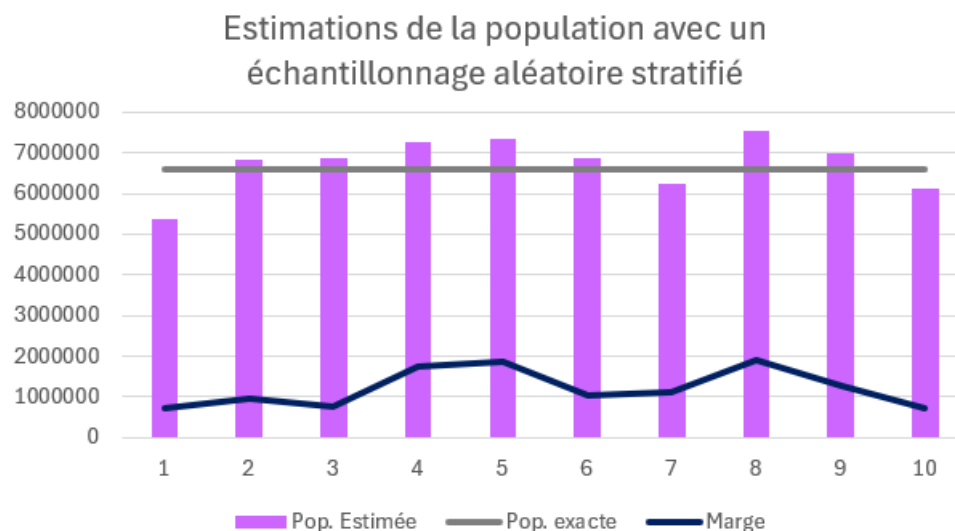
```
  # Borne inférieure de l'IDC pour T  
  binf <- idcmoy[1]*N
```

```
  # Borne supérieure de l'IDC pour T  
  bsup <- idcmoy[2]*N
```

```
  # IDC pour T  
  idcT <- c(binf, bsup)
```

```
  #Marge d'erreur  
  marge <- (idcT[2]-idcT[1])/2
```


Comme pour l'échantillonnage aléatoire simple, on exporte les données dans un csv et nous réalisons un graphique.



La stratification améliore la précision des estimations. En divisant la population en sous-groupes homogènes (strates) avant de tirer aléatoirement, on réduit la variance : la plus petite barre atteint les 5.4M et la plus haute les 7.6M, toutes ne s'éloignent donc pas de la vraie valeur. La marge, quant à elle, reste plus contenue et régulière que pour les estimations précédentes.

Conclusion

Cette partie nous a permis de mettre en pratique deux grandes méthodes d'échantillonnage sur des données réelles de l'INSEE triée sur la région Nouvelle-Aquitaine, puis d'en comparer les performances. Le sondage aléatoire simple a montré ses limites face à une population fortement hétérogène. En l'absence de structure dans le tirage, la présence ou l'absence de grandes villes dans l'échantillon suffit à faire varier l'estimation de plusieurs millions d'habitants, produisant des intervalles de confiance larges et peu fiables. Le sondage stratifié a quant à lui démontré une bien meilleure fiabilité. En regroupant les communes en strates homogènes selon leur taille de population (et en rendant exhaustive la strate des grandes villes), on neutralise les principales sources de variabilité. Les estimations obtenues se situent systématiquement dans une fourchette plus étroite autour de la vraie valeur, et les marges d'erreur sont sensiblement réduites et plus stables d'un tirage à l'autre. Ces résultats confirment un principe fondamental de la théorie des sondages : plus la population est hétérogène, plus la stratification est indispensable pour obtenir des estimations précises sans augmenter démesurément la taille de l'échantillon. Dans des contextes réels (études de marché, enquêtes nationales...), le choix de la méthode d'échantillonnage et la définition des strates sont donc des étapes déterminantes pour la qualité de l'inférence statistique.

Traitement de données d'enquête

Dans cette seconde partie, nous nous intéressons aux données d'une enquête menée auprès d'étudiants sur la pratique du sport. Ces données, déjà exploitées lors d'une SAÉ précédente, vont cette fois être analysées sous un angle inférentiel.

L'objectif est de déterminer s'il existe des relations statistiquement significatives entre la variable « sport » et d'autres variables qualitatives de notre choix. Pour cela, nous utiliserons le test du khi-deux d'indépendance, qui permet de vérifier si deux variables qualitatives sont liées ou si leur association est due au simple hasard.

Test d'indépendance du khi-deux

Dans un premier temps, nous importons les données issues du csv, toujours avec la même méthode, puis nous filtrons les lignes en enlevant celles qui ont des NA / cellule vide dans la colonne « sport ».

```
#Nettoyage : suppression des lignes sans réponse à "sport"  
tableau <- tableau[!is.na(tableau$sport) & tableau$sport != "", ]
```

Puis nous réalisons des tableaux de contingences, nous avons choisi les variables : santé, fumer, alimentation et fan (de sport).

```
#Croisement sport / santé  
tab1 <- table(sport = tableau$sport, sante = tableau$sante)
```

Ensuite on calcule les effectifs théoriques afin de vérifier qu'ils sont tous bien supérieur à 5 ce qui nous permet de calculer tous les khi-deux.

```
#Effectifs théoriques (conditions de validité du khi-deux)  
chisq.test(tab1)$expected  
...  
  
#Tests du khi-deux : sport vs chaque variable  
khideux_tab1 <- chisq.test(tab1)  
...
```

Puis on crée le tableau synthétique :

```
#Tableau synthétique des p-valeurs  
#Noms des variables testées contre "sport"  
variables <- c("sante", "fumer", "alimentation", "fan")  
  
#Regroupement des résultats des tests dans une liste  
tests <- list(khideux_tab1, khideux_tab2, khideux_tab3, khideux_tab4)  
|  
#Extraction des p-valeurs et statistiques khi-deux arrondies à 4 décimales  
pvaleurs <- sapply(tests, function(t) round(t$p.value, 4))  
khi2 <- sapply(tests, function(t) round(t$statistic, 4))
```

```

#Liste des tableaux de contingence
tabs <- list(tab1, tab2, tab3, tab4)

#Fonction de calcul du V de Cramer
v_cramer <- function(tab) {
  test <- chisq.test(tab)
  round(sqrt(test$statistic / (sum(tab) * (min(dim(tab)) - 1))), 4)
}

#Calcul du V de Cramer uniquement pour les tests significatifs (NA sinon)
v_values <- ifelse(pvaleurs < 0.05, sapply(tabs, v_cramer), NA)

#Tableau final unifié : khi-deux + p-valeur + significativité + V de Cramer
tableau_final <- data.frame(
  Variable = variables,
  khi2 = khi2,
  P_valeur = pvaleurs,
  Significatif = ifelse(pvaleurs < 0.05, "Oui", "Non"),
  V_Cramer = v_values
)

#Affichage du tableau et de la liaison la plus forte
print(tableau_final, row.names = FALSE)
cat("Liaison la plus forte :", variables[which.max(v_values)], "\n")

```

Variable	Khi2	P_valeur	Significatif	V_Cramer
sante	0.3093	0.5781	Non	NA
fumer	0.3805	0.5373	Non	NA
alimentation	13.8016	0.0002	Oui	0.1921
fan	75.2727	0.0000	Oui	0.4486

Conclusion

Cette analyse nous a permis d'étudier statistiquement les relations entre la pratique du sport et quatre variables qualitatives : la santé, le tabac, l'alimentation et le fait d'être fan de sport.

Les tests du khi-deux ont révélé que certaines de ces variables entretiennent un lien significatif avec la pratique sportive, c'est-à-dire que leur association ne peut pas être attribuée au simple hasard. Pour les relations significatives identifiées, le V de Cramer a permis de quantifier l'intensité de ces liens.

La liaison la plus forte est celle entre la pratique du sport et le fait d'être fan de sport. Ce résultat est cohérent : un étudiant qui suit et apprécie le sport en tant que spectateur a davantage tendance à en pratiquer lui-même, ce qui traduit un intérêt global pour l'activité sportive.

Ces résultats confirment que la pratique du sport ne s'inscrit pas isolément dans le mode de vie des étudiants : elle est associée à d'autres comportements et habitudes. Toutefois, il convient de rester prudent dans l'interprétation : le test du khi-deux établit l'existence d'un lien entre deux variables, mais ne permet pas de conclure à un lien de causalité. Des facteurs extérieurs non mesurés dans cette enquête pourraient également expliquer les associations observées.

Code R

Échantillonnage aléatoire simple

Pour 10 échantillons

```
#Définition du répertoire
setwd("E:\\SAE Echantillonnage et estimation2")
#Chargement de la bibliothèque sampling
library(sampling)
#Lecture du fichier CSV
tableau <- read.csv2("population_francaise_communes.csv")
#Extraction des communes de Nouvelle-Aquitaine avec les colonnes utiles
donnees <- tableau[tableau$Nom.de.la.region == "Nouvelle-Aquitaine",
                    c("Code.département", "Commune", "Population.totale")]
#Affiche les 6 premières communes
head(donnees)
#U : vecteur des noms de communes
U <- donnees$Commune
#N : taille de la population (nombre total de communes)
N <- length(U)
#T : total réel de la population
T <- sum(donnees$Population.totale)
#Tirage d'un échantillon aléatoire simple sans remise
#Tirage des 10 échantillons
for (i in 1:10){
  #Tirage aléatoire de 100 communes
  n <- 100
  E <- sample(U, n)
  #Données des 100 communes tirées
  donnees2 <- donnees[donnees$Commune %in% E, ]
  #Sous-ensemble limité aux colonnes Commune et Population.totale
  donnees3 <- subset(donnees2, select = c(Commune, Population.totale))

  #Moyenne de la population sur cet échantillon
  xbar <- mean(donnees3$Population.totale, na.rm = TRUE)
  #IDC de la moyenne (test t à 95 %)
  idcmoy <- t.test(donnees2$Population.totale)$conf.int
  #Estimation du total de la population
  T_est <- N * xbar
  #IDC du total estimé
  idcT <- idcmoy * N
  #Marge d'erreur associée à cet échantillon
  marge <- (idcT[2] - idcT[1]) / 2
  #Ajout d'une ligne au tableau : total réel, total estimé, marge d'erreur
  new_row <- data.frame(Pop_exacte = T,
                        Pop_est = T_est,
                        Marge = marge,
                        IDC_inf = idcT[1],
                        IDC_sup = idcT[2])
  resultats <- rbind(resultats, new_row)
}
#Affichage du tableau récapitulatif des 10 simulations
print(resultats)
#Export des résultats dans un fichier CSV (séparateur point-virgule)
write.csv2(resultats, "resultats_simple.csv", row.names = FALSE)
```

Pour un échantillon

```
#Sélection aléatoire de n = 100 communes
parmi U
n <- 100
E <- sample(U, n)
head(E)
#Récupération des lignes correspondant aux
communes échantillonnées
donnees1 <- donnees[donnees$Commune
%in% E, ]
head(donnees1)
#Estimation de la moyenne et intervalle de
confiance
#Moyenne de la population totale sur
l'échantillon (valeurs manquantes ignorées)
xbar <- mean(donnees1$Population.totale,
na.rm = TRUE)
#Intervalle de confiance à 95 % de la moyenne
via le test t
idcmoy <-
t.test(donnees1$Population.totale)$conf.in
#Estimateur du total : moyenne échantillonnale
× taille de la population
T_est <- N * xbar
#Transposition de l'IDC de la moyenne en IDC
du total (multiplication par N)
idcT <- idcmoy * N
#Marge d'erreur
marge <- (idcT[2] - idcT[1]) / 2
#Initialisation du tableau de résultats
resultats <- data.frame(
  Pop_exacte = numeric(),
  Pop_est = numeric(),
  IDC_inf = numeric(),
  IDC_sup = numeric(),
  Marge = numeric()
)
```

Échantillonnage aléatoire stratifié

```
# Définit le répertoire de travail
setwd("E:\\SAE Echantillonnage et estimation2")
# Charge le package d'échantillonnage
library(sampling)
# Chargement et préparation des données
# Lit le fichier CSV
tableau <- read.csv2("population_francaise_communes.csv")
# Filtre la région Nouvelle-Aquitaine et garde ces 3 colonnes
donnees <- tableau[tableau$Nom.de.la.region == "Nouvelle-Aquitaine",
  c("Code.département", "Commune", "Population.totale")]
# Convertit la population en numérique
donnees$Population.totale <- as.numeric(donnees$Population.totale)
# Supprime les lignes sans population
donnees <- donnees[!is.na(donnees$Population.totale), ]
# Affiche min, max, moyenne, quartiles
summary(donnees$Population.totale)
# Total réel de la population (valeur de référence)
T <- sum(donnees$Population.totale)
# Création des strates (9 strates)
# Découpe la population en classes selon les bornes définies et numérote les strates de 1 à 9
donnees$strate <- cut(donnees$Population.totale,
  breaks = c(0, 163, 239, 349, 507, 720, 1112, 2185, 20000, Inf),
  labels = c(1,2,3,4,5,6,7,8,9))
# Garde uniquement les colonnes utiles
donneesstrat <- donnees[, c("Commune", "Population.totale", "strate")]
# Affiche les 6 premières lignes pour vérification
head(donneesstrat)
# Paramètres de l'échantillonnage
# Trie par strate (obligatoire pour strata())
data <- donneesstrat[order(donneesstrat$strate),]
# Nombre de communes par strate (Nh)
Nh <- table(data$strate)
# Nombre total de communes (N)
N <- sum(Nh)
# Poids de chaque strate = Nh / N
gh <- Nh/N
# Initialisation du tableau de résultats
# Crée un tableau vide pour stocker les résultats des 10 tirages
resultats <- data.frame(
  Pop_exacte = numeric(),
  Pop_est = numeric(),
  Marge = numeric(),
  IDC_inf = numeric(),
  IDC_sup = numeric()
)
# Boucle : 10 tirages indépendants
for (i in 1:10){
  # Taille souhaitée de l'échantillon (hors strate 9 exhaustive)
  n <- 100
  # Allocation proportionnelle : nh = n * Nh/N, avec minimum 1 par strate
  nh <- pmax(round(c(n*Nh[1]/N, n*Nh[2]/N, n*Nh[3]/N, n*Nh[4]/N,
    n*Nh[5]/N, n*Nh[6]/N, n*Nh[7]/N, n*Nh[8]/N, n*Nh[9]/N)), 1)
  # Strate 9 exhaustive : toutes les grandes villes sont incluses
  nh[9] <- Nh[9]
  # Taux de sondage par strate = nh / Nh
  fh <- nh/Nh
  # Tirage stratifié avec remise dans chaque strate
  st <- strata(data, stratanames = c("strate"), size = nh, method = "srswr")
  # Extrait les données des communes tirées
```

```

data1 <- getdata(data, st)
# Extraction des sous-échantillons par strate
ech1 <- data1[data1$strate==1, ]
ech2 <- data1[data1$strate==2, ]
ech3 <- data1[data1$strate==3, ]
ech4 <- data1[data1$strate==4, ]
ech5 <- data1[data1$strate==5, ]
ech6 <- data1[data1$strate==6, ]
ech7 <- data1[data1$strate==7, ]
ech8 <- data1[data1$strate==8, ]
ech9 <- data1[data1$strate==9, ]
# Moyennes de population par strate
m1 <- mean(ech1$Population.totale)
m2 <- mean(ech2$Population.totale)
m3 <- mean(ech3$Population.totale)
m4 <- mean(ech4$Population.totale)
m5 <- mean(ech5$Population.totale)
m6 <- mean(ech6$Population.totale)
m7 <- mean(ech7$Population.totale)
m8 <- mean(ech8$Population.totale)
m9 <- mean(ech9$Population.totale)
# Variances de population par strate
var1 <- var(ech1$Population.totale)
var2 <- var(ech2$Population.totale)
var3 <- var(ech3$Population.totale)
var4 <- var(ech4$Population.totale)
var5 <- var(ech5$Population.totale)
var6 <- var(ech6$Population.totale)
var7 <- var(ech7$Population.totale)
var8 <- var(ech8$Population.totale)
# Variance = 0 si 1 seul élément (strate exhaustive)
var9 <- if (length(ech9$Population.totale) < 2) 0 else var(ech9$Population.totale)
# Moyenne stratifiée = somme pondérée des moyennes par strate
Xbarst <- (Nh[1]*m1 + Nh[2]*m2 + Nh[3]*m3 + Nh[4]*m4 + Nh[5]*m5 +
          Nh[6]*m6 + Nh[7]*m7 + Nh[8]*m8 + Nh[9]*m9) / N
# Variance de l'estimateur : somme des contributions de chaque strate
varXbarst <- (gh[1]^2)*(1-fh[1])*var1/nh[1] + (gh[2]^2)*(1-fh[2])*var2/nh[2] + (gh[3]^2)*(1-fh[3])*var3/nh[3] + (gh[4]^2)*(1-
fh[4])*var4/nh[4] + (gh[5]^2)*(1-fh[5])*var5/nh[5] + (gh[6]^2)*(1-fh[6])*var6/nh[6] + (gh[7]^2)*(1-fh[7])*var7/nh[7] + (gh[8]^2)*(1-
fh[8])*var8/nh[8] + (gh[9]^2)*(1-fh[9])*var9/nh[9]
# Seuil de signification
alpha <- 0.05
# Total estimé = N * moyenne stratifiée
Tstr <- N * Xbarst
# Borne inférieure de l'IDC pour le total T
binf <- idcmoy[1]*N
# Borne supérieure de l'IDC pour le total T
bsup <- idcmoy[2]*N
# IDC pour T
idcT <- c(binf, bsup)
# Marge d'erreur = demi-largeur de l'IDC
marge <- (idcT[2]-idcT[1])/2
# Ajout des résultats du tirage i dans le tableau
resultats <- rbind(resultats, data.frame(
  Pop_exacte = T,
  Pop_est   = Tstr,
  Marge     = marge,
  IDC_inf   = idcT[1],
  IDC_sup   = idcT[2]
))
}
# Affichage et export
# Affiche le tableau des 10 résultats dans la console

```

```

print(resultats)
# Exporte en CSV séparateur ";" sans numéro de ligne
write.csv2(resultats, "resultats3.csv", row.names = FALSE)

```

Test d'indépendant du khi-deux

```

#Définition du répertoire de travail
setwd("D:\\SAE Echantillonnage et estimation")
#Chargement des données
tableau <- read.csv2("EnqueteSportEtudiant2024.csv")
#Aperçu des données (décommenté si besoin)
head(tableau)
#Nettoyage : suppression des lignes sans réponse à "sport"
tableau <- tableau[!is.na(tableau$sport) & tableau$sport != "", ]
#TABLEAUX DE CONTINGENCE : SPORT VS CHAQUE VARIABLE QUALITATIVE
#Croisement sport / santé
tab1 <- table(sport = tableau$sport, sante = tableau$sante)
#Croisement sport / tabac
tab2 <- table(sport = tableau$sport, fumer = tableau$fumer)
#Croisement sport / alimentation
tab3 <- table(sport = tableau$sport, alimentation = tableau$alimentation)
#Croisement sport / fan de sport
tab4 <- table(sport = tableau$sport, fan = tableau$fan)
#Effectifs théoriques (conditions de validité du khi-deux)
#Toutes les cellules doivent idéalement être >= 5
chisq.test(tab1)$expected
chisq.test(tab2)$expected
chisq.test(tab3)$expected
chisq.test(tab4)$expected
#Tests du khi-deux : sport vs chaque variable
khideux_tab1 <- chisq.test(tab1)
khideux_tab2 <- chisq.test(tab2)
khideux_tab3 <- chisq.test(tab3)
khideux_tab4 <- chisq.test(tab4)
#Tableau synthétique des p-valeurs
#Noms des variables testées contre "sport"
variables <- c("sante", "fumer", "alimentation", "fan")
#Regroupement des résultats des tests dans une liste
tests <- list(khideux_tab1, khideux_tab2, khideux_tab3, khideux_tab4)
#Extraction des p-valeurs et statistiques khi-deux arrondies à 4 décimales
pvaleurs <- sapply(tests, function(t) round(t$p.value, 4))
khi2 <- sapply(tests, function(t) round(t$statistic, 4))
#Liste des tableaux de contingence
tabs <- list(tab1, tab2, tab3, tab4)
#Fonction de calcul du V de Cramér
v_cramer <- function(tab) {
  test <- chisq.test(tab)
  round(sqrt(test$statistic / (sum(tab) * (min(dim(tab)) - 1))), 4)
}
#Calcul du V de Cramér uniquement pour les tests significatifs (NA sinon)
v_values <- ifelse(pvaleurs < 0.05, sapply(tabs, v_cramer), NA)
#Tableau final unifié : khi-deux + p-valeur + significativité + V de Cramér
tableau_final <- data.frame( Variable = variables, Khi2 = khi2, P_valeur = pvaleurs, Significatif = ifelse(pvaleurs < 0.05, "Oui", "Non",
V_Cramer = v_values)
#Affichage du tableau et de la liaison la plus forte
print(tableau_final, row.names = FALSE)
cat("Liaison la plus forte :", variables[which.max(v_values)], "\n")

```